A CASE STUDY IN AI FOR TECHNICAL DOCUMENTATION

From raw notes to maintained docs

Designing and testing a seven-agent AI pipeline for the full documentation lifecycle

Intake · Planning · Drafting · Review · QA · Publishing · Maintenance

Debargho Ghosh
Technical Writer · 16 years of internal and external documentation
debarghoghosh86@gmail.com
linkedin.com/in/debargho-ghosh-8566ab23

November 2026

Product names in all test material are fictional (Contoso Charts).

Contents

1 Summary . . . 3
2 The challenge . . . 4
3 The pipeline . . . 5
4 Design principles . . . 6
5 How I tested it . . . 7
6 A worked example . . . 8
7 The seven agents . . . 10
8 Results . . . 12
9 Lessons learned . . . 13
10 Limits and next steps . . . 14
About the author . . . 15
Appendix A: The shared house style . . . 16

AT A GLANCE

1 Summary

I'm a technical writer with 16 years of experience in internal and external documentation. To build practical skills in AI, I designed, tested, and connected **seven AI agents**, one for each stage of the documentation lifecycle: intake, planning, drafting, review, QA, publishing, and maintenance. They run as a pipeline in Claude Projects, without code, with a writer approving the work at six checkpoints.

Each agent was tested on realistic material with traps planted on purpose, against an answer key written before the run. When an agent failed, I changed its instructions, checked what else the change affected, and ran it again. The pipeline went through 21 instruction versions before it was finished.

27 → 0
Style issues: a legacy hand-written help page vs. the pipeline's final draft

0
Invented facts across all drafting runs. Every drafted fact is tagged to a source.

21
Instruction versions across seven agents, each one driven by a test result

10 / 10
Planner: planted structure traps found, including a duplicate article

10 + 1
QA: planted release defects found, plus one I hadn't planted. No false alarms.

6 / 6
Publisher: not-ready items held back, including a fix claimed but not made

The most important finding

The model's style knowledge was good from the start. What made the pipeline trustworthy was an **editor's judgment**, written down as rules: one comment per sentence, don't fix what you've queried, keep a title so links don't break, ask the SME instead of guessing, and never publish what isn't ready. Experienced writers already know these habits. The project shows they can be turned into working AI systems.

See for yourself. One full run, with the agents' real outputs at every stage, is included in this report (section 6) and in the portfolio's `examples/` folder.

WHY THIS PROJECT

2 The challenge

Documentation work is mostly judgment. A writer sorts what was decided from what was only discussed, notices what's missing, keeps dozens of articles consistent with each other, and decides what's safe to publish. Generating text is the easy part.

So the question I wanted to answer wasn't "Can AI write docs?" It was:

> *How much of a documentation team's judgment can an AI pipeline apply reliably, and what does a senior writer have to put into it to make it trustworthy?*

What I set out to build

- **The whole lifecycle,** not just drafting: from messy source material to published files, and back again when the product changes.
- **One agent per stage,** each with a clear job, rules, and an output the next stage can use directly.
- **A writer in charge.** The agents prepare and check; a human approves at every checkpoint.
- **Evidence, not claims.** Every agent tested against planted traps, with results recorded version by version.

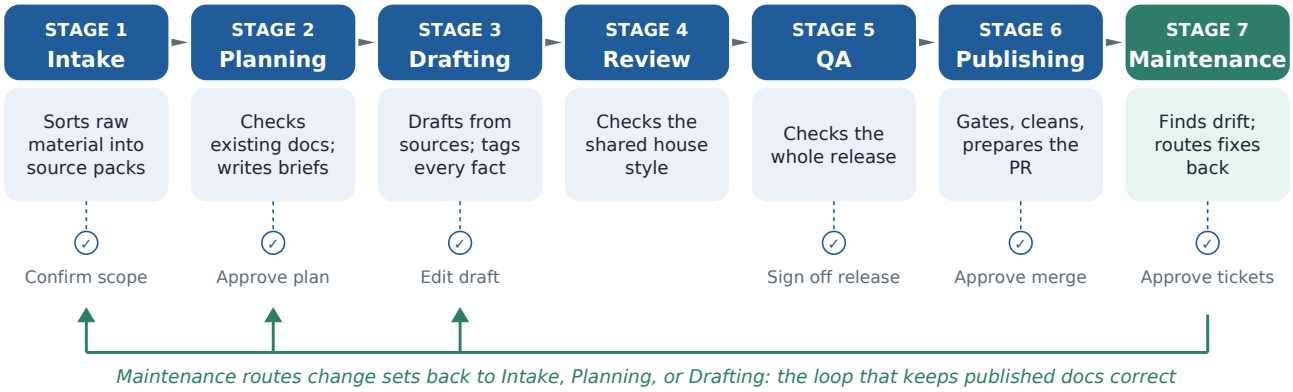
Constraints

- No code. Every agent is a set of written instructions in a Claude Project, which made the rules themselves the product.
- Fictional test material. Product names are placeholders ("Contoso Charts"). One test page came from my own past work, with the product name removed.

HOW IT FITS TOGETHER

3 The pipeline

Seven agents, six human checkpoints, and a loop that sends post-release changes back into the pipeline.



Maintenance routes change sets back to Intake, Planning, or Drafting: the loop that keeps published docs correct

✓ = human checkpoint. Planner, drafter, reviewer, and QA share one rules file: house-style.md

Stage	Agent	Takes in	Produces	Human checkpoint
1. Intake	Doc intake v3	Chat threads, meeting notes, tickets, pull requests, support email	Doc impact list, source packs with evidence, decision log, questions (I-Q#)	Confirm scope
2. Planning	Doc planner v2	Intake output + inventory of existing docs	Doc plan, drafting briefs with outlines, link map (P-Q#)	Approve plan
3. Drafting	Doc drafter v7	Brief + source pack	Draft with [S#] tags and [GAP] markers (D-Q#)	Edit draft
4. Review	Style guide reviewer v5	Draft + house-style.md	Prioritized issues table, image checks, open GAPs	—
5. QA	Doc QA v1	All drafts + plan + link map + source pack	Release verdicts, link check, terminology table, blocking questions	Sign off release
6. Publishing	Doc publisher v1	QA report + final drafts + slug map	Markdown files, patches, pull request description, publication record	Approve merge
7. Maintenance	Doc maintenance v2	New changes + published docs	Change sets with shared wording, routed back (Ready / Hold / Blocked)	Approve tickets

WHAT MAKES IT TRUSTWORTHY

4 Design principles

Traceable facts

Every drafted sentence carries a source tag ([S1], [S2] ...) that points to a block in the source pack, and every source pack fact carries its evidence, such as (R5, Oct 13). A reviewer can check any sentence in seconds. Missing information becomes a [GAP] marker and an SME question, never a guess.

Shared contracts between agents

When one agent's output is another's input, they must agree on the format, just as writers and editors agree on markup. The agents share markers, a source pack format, a drafting brief format, and one numbered question list: I-Q# from intake, P-Q# from planning, D-Q# from drafting. The SME gets one list, and nothing is asked twice.

A single source of truth

One rules file, house-style.md, is shared by four agents. When the drafter and the reviewer each had their own copy of a rule, they disagreed within a day. A rule changed once now applies everywhere, the same idea as content reuse in DITA.

Gates with the fewest permissions

QA gives each draft a release verdict. The publisher may only remove source tags and build links; any other change sends the file back to a person. It verifies claimed fixes against the text instead of trusting them.

A closed loop

After publishing, maintenance compares new changes against the live docs, groups related fixes into change sets with one shared wording, and routes them back to intake, planning, or drafting. Changes for an unreleased version wait until it ships.

Humans decide

The agents never approve their own work. Scope, plan, drafts, release, merge, and maintenance tickets all need a person's sign-off. Agents flag risks and make recommendations; the docs lead decides.

METHOD

5 How I tested it

Each agent was built the same way: write the instructions, test them on realistic input with planted traps, score the output against an answer key, fix what failed, and check whether the fix broke anything else.

The evaluation loop

Step	What it means in practice
1. Write the answer key first	List every trap and the correct handling before the first run, so "looks good" becomes a score
2. Plant realistic traps	Reversed decisions, Won't Fix tickets, customer names, duplicate articles, wrong link titles, claimed-but-missing fixes
3. Run and score	Compare the output with the answer key, and note anything the agent caught that the key missed
4. Fix the instructions	Change the rule that caused the failure, usually by adding a bad and a good example
5. Test the whole pipeline	A fix in one agent can break something two stages later, so rerun the stages downstream

Instruction versions

Agent	Versions	What drove the changes
Style guide reviewer	5	Cluttered tables, inconsistent grouping, misquotes, missing rules, then moving rules into house-style.md
Doc drafter	7	Missed SME questions, misplaced GAPs, repeated questions, briefs, a Troubleshooting template, an over-strict source rule
Doc intake	3	Internal context leaking into source packs, superseded citations, then a lost version number and colliding question numbers
Doc planner	2	A summary that miscounted its own table; question lists missing from briefs
Doc maintenance	2	Inconsistent routing; related fixes worded separately
Doc QA	1	Passed on the first version
Doc publisher	1	Passed on the first version
house-style.md	2	A Troubleshooting exception for the verb-first heading rule

Real legacy content

The reviewer was also tested on an installation page I wrote years ago, reviewed as HTML so it could check images and links. The first version found 23 issues in a cluttered table; the third found 27 issues, cleanly grouped, with a separate check of all 14 images. Along the way it caught things I'd missed: screenshots that appeared to be out of order, and a disk-space unit ("Mib") that technically meant mebibits.

SEE FOR YOURSELF

6 A worked example

One run of the pipeline for a fictional feature, "Refresh from Excel" in Contoso Charts 4.3. The excerpts below are the agents' real outputs, copied word for word.

The raw input

Six messy sources, written as a test scenario: a Slack thread where the plan changes, meeting notes with an open question, a Done ticket and a Won't Fix ticket, a pull request, and a support email. An excerpt:

```
Oct 2, Arjun (PM): Kicking off "Refresh from Excel" for 4.3. ...
                   Plan is auto-refresh every 5 minutes.
Oct 4, Arjun (PM): Ugh. Fabrikam is the customer pushing for this, they'll want auto.
Oct 9, Arjun (PM): Decision: 4.3 ships MANUAL refresh only.
                   Auto-refresh moves to 4.4, maybe.

Meeting notes, Oct 10:
- Requires the linked workbook to be closed in Excel? Mei wasn't sure.

Pull request, Oct 13:
If the workbook is open in Excel, refresh reads the last SAVED version.
```

Stage 1: Intake applies the final decision and keeps the evidence

Intake used the Oct 9 decision, logged the reversed one, removed the customer's name, and noticed that the pull request answered the question left open in the meeting notes. From its source pack:

```
[S2] Requirements
- Refresh works only for charts linked to an Excel workbook (.xlsx). (R2, Oct 10)
- If the linked workbook is open in Excel, refresh reads the last saved
  version of the workbook. Unsaved changes aren't included. (R5, Oct 13)
```

Decision log: topic	Earlier decision (source)	Final decision (source)
Refresh mode	Auto-refresh every 5 minutes (R1, Oct 2)	Manual refresh only (R1, Oct 9)
Workbook open in Excel	Unknown whether the workbook must be closed (R2, Oct 10)	Refresh works while the workbook is open and reads the last saved version (R5, Oct 13)

Stage 2: Planning avoids a duplicate

Intake proposed a new article. The planner checked 14 existing articles, found a 2021 article that already covered the task with an outdated workaround, and rewrote it in place instead, keeping its title so links wouldn't break. It also found a troubleshooting page, which intake couldn't see, still claiming charts "update automatically every 5 minutes."

Stage 3: Drafting tags every sentence

```
## Before you begin

- The chart must be linked to an Excel workbook (.xlsx). [S2] For more
  information, see Link a chart to Excel data.
- If the workbook is open in Excel, save it before you refresh. Refresh reads
  the last saved version and doesn't include unsaved changes. [S2]

## Refresh all linked charts in a presentation

- On the **Contoso Charts** tab, in the **Data** group, select
  **Refresh All Charts**. Every linked chart in the presentation refreshes. [S3]
  [GAP: what happens to charts that aren't linked — see I-Q4]
```

Stage 4: Review

The reviewer checked the draft against all 23 house-style rules and found **0 issues**. It listed the four open GAPs to resolve before publishing, and explained three borderline sentences it had checked and accepted. On the second article, a troubleshooting page, it found 3 issues in 7 places, including a section heading that restated the symptom instead of naming a cause.

Stage 5: QA checks the whole release

QA read all the drafts together and found every planted defect, plus a missing link that wasn't planted:

Draft	Problem found	Verdict
Charts don't update after editing Excel	Links to intake's old title; "Refresh All" instead of "Refresh All Charts"	Blocked
Error messages	Message text missing "the"; a fix with no source; a missing link (not planted)	Blocked
What's new	Claims CSV support, contradicting the .xlsx-only source; "Version 4.3" without the product name	Ready after fixes
Link a chart to Excel data	Planned but never drafted	Missing

Stages 6 and 7

The publisher published two items and held back six, including a draft whose writer said a fix was applied when it wasn't. After the release, maintenance found that a single button rename affected five published articles, and grouped the fix into one change set with one shared wording.

STAGE BY STAGE

7 The seven agents

Stage 1 · Doc intake (v3)

Risk it's designed around: Passing along the wrong facts: a reversed decision, a Won't Fix feature, a plan presented as fact, or a customer's name.

Key rules

- The latest decision wins; earlier ones go in a decision log
- Only what ships; speculation ("maybe," "next year") stays out
- Confidential details removed completely
- Every fact keeps its evidence; the reason for a doc stays out of source packs
- A version block in every source pack; questions numbered I-Q#

Result: Six planted traps, all handled from v1, including noticing that a later source answered an earlier open question. v2 and v3 fixed internal context leaking into source packs and a version number lost two stages later.

Stage 2 · Doc planner (v2)

Risk it's designed around: Publishing a duplicate article, leaving a contradicting page live, or putting a topic in the wrong place.

Key rules

- Check existing docs before planning anything new
- One action per doc: New, Update, Rewrite, Merge, Retire, or No change
- Find stale content intake couldn't see
- Link to existing articles that answer open questions
- Drafting briefs with outlines mapped to source facts

Result: Eight planted structure traps: 10 of 10 checks passed. It also noticed the drafter had no Troubleshooting template. v2 fixed a summary that miscounted its own table.

Stage 3 · Doc drafter (v7)

Risk it's designed around: Inventing facts: a UI label, a limit, or a step that sounds right but isn't in any source.

Key rules

- Use only facts from the sources, and tag every one
- Mark missing information as [GAP]; flag disagreements as [CONFLICT]
- Leave out code names, ticket numbers, people, and roadmap plans
- An SME checklist: platforms, versions, permissions, errors, limits, result
- Task, Concept, Reference, and Troubleshooting templates

Result: No invented facts in any run. The planted source conflict (50 vs. 20 charts) was flagged, not resolved. When the template asked for a reason no source gave, it left it out and said why.

Stage 4 · Style guide reviewer (v5)

Risk it's designed around: Reviews that are inconsistent, cluttered, or that "fix" things the writer should decide.

Key rules

- 23 rules in the shared house-style.md, each with bad and good examples
- One row per sentence, one combined fix; repeated patterns grouped
- "Pending Q#" instead of a fix that depends on an open question
- A severity scale set by my editorial judgment
- A separate accessibility check for every image

Result: On a legacy page: 27 well-organized issues and 14 image checks. On the pipeline's draft: 0 issues.

Stage 5 · Doc QA (v1)

Risk it's designed around: Defects that live between documents: wrong link titles, contradicting facts, a planned article nobody wrote.

Key rules

- Completeness against the plan
- Links checked against the link map, with exact titles
- Terminology and facts compared across articles
- Unsourced facts and exact message text
- A release verdict for each draft

Result: Ten planted defects found with no false alarms, plus one defect that wasn't planted.

Stage 6 · Doc publisher (v1)

Risk it's designed around: Letting something through that isn't ready, or quietly changing what a writer wrote.

Key rules

- Publish only Ready items; verify every claimed fix against the text
- Block files with [GAP], internal question numbers, TBD, or TODO
- Remove only source tags and evidence notes
- Updates become patches; links come from the site's slug map
- Flag release risks; record what was published

Result: Nine traps: 2 items published, 6 held back, 0 redirects invented. The publication record handed the next job to maintenance.

Stage 7 · Doc maintenance (v2)

Risk it's designed around: Docs drifting after publishing: one change breaking several articles, or SME answers never reaching the docs.

Key rules

- Sweep every article for a changed label, message, or limit
- SME answers close the GAPs waiting on them; support tickets count
- Earlier release notes stay unchanged; add version notes
- Group related fixes into change sets with one shared wording
- Hold changes until their release ships

Result: Seven changes: every planted drift found, including a rename in five articles. v2 grouped 21 fixes into 7 change sets: 4 on hold for the release and 3 ready now.

EVIDENCE

8 Results

Agent	Test	Result
Intake	Raw input with 6 planted traps	All traps handled: final decision used, Won't Fix and speculation excluded, no confidential details, a resolved question noticed, a wrong article flagged
Planner	Intake output + 14 existing articles, 8 traps	10 of 10 checks: no duplicate article, stale page found, existing articles used to answer questions
Drafter	Source pack with a conflict, 3 gaps, 5 internal items	0 invented facts; conflict flagged; 3 of 3 gaps asked about (from v2); 5 of 5 internal items left out
Reviewer	Planted-error sample and a real legacy page	Legacy page: 27 issues, 14 images checked. Pipeline draft: 0 issues
QA	Release of 5 drafts with 10 planted defects	10 of 10 found, plus 1 not planted; no false alarms
Publisher	Final release with 9 traps	2 published, 6 held back, 0 redirects invented; claimed-but-missing fix caught
Maintenance	7 changes after release	All drift found, including a 5-article rename; 7 change sets with shared wording

What the agents caught that I didn't

- Screenshots in my old help page that appeared to be out of order.
- A missing link in a release draft that wasn't in my answer key.
- Approved release notes that left out a UI rename shipping in the same version.
- A question every lead writer should ask: did linked charts *ever* update automatically? The answer decides whether the release note describes a change or a correction.

WHAT I'D BRING TO A TEAM

9 Lessons learned

01 The writer's expertise is the product.
The model knew the style rules. What it needed was an editor's judgment about output and process.

02 Every instruction has side effects.
A rule meant to remove roadmap hints also removed the version number from the draft, two stages later. Test the whole pipeline, not just the agent you changed.

03 Strict rules can remove good content.
"Use only the listed sources" made the drafter leave out a step's result. Clear rules need room for judgment.

04 Agents need shared contracts.
Markers, formats, and numbered questions are the interfaces between stages. Without them, each agent guesses.

05 Single-source the rules.
Two copies of a style guide drift apart, in agents as with writers.

06 An agent needs to see the existing docs.
Without an inventory, intake planned an article that already existed.

07 Check the release, not just the article.
The worst defects live between documents.

08 Verify, don't trust.
A fix marked as applied wasn't. One check against the text caught it.

09 Group related fixes.
Five separate tickets for one rename risk five different wordings.

10 Write the answer key first, but don't treat it as the ceiling.
The agents caught real problems the key had missed.

11 Runs vary.
The same input can produce different, defensible plans. That's why every stage ends with a human checkpoint.

12 Know when to stop.
"Good enough to use" beats endless tuning.

HONEST SCOPE

10 Limits and next steps

Limits of this work

- **Test scenarios, not production.** The inputs were realistic but written by me, with traps planted on purpose. Real material will be messier.
- **Mostly single runs.** Outputs vary between runs; a fuller evaluation would run each test several times and record the spread.
- **Manual handoffs.** I moved each output to the next agent myself. Nothing is automated yet.
- **Some inputs were incomplete by design.** QA couldn't check heading order without every brief, and the publisher couldn't confirm a release risk without the live text of a held article. Both are input fixes, not instruction fixes.

Next steps

Phase	Goal
A. Agents without code	Done: seven agents designed, tested, and connected
B. Visual automation	Connect the stages with a workflow tool (n8n or Make), so each output moves to the next stage automatically
C. Code with AI assistance	Rebuild the pipeline in Python, run it from GitHub pull requests, and measure it across a larger test set with several runs per test

Skills demonstrated

AI agent and prompt design · AI output evaluation · information architecture · content strategy · docs-as-code conventions (front matter, pull requests, redirects) · content governance and release management · version-aware documentation · accessibility (WCAG alt text and link text) · Microsoft Writing Style Guide · SME collaboration

About the author

Debargho Ghosh is a technical writer with 16 years of experience producing internal and external documentation. This project is part of his move into AI for documentation: designing agents that apply a writer's judgment, and evaluating them with the same care as any published doc.

Email	debarghoghosh86@gmail.com
LinkedIn	linkedin.com/in/debargho-ghosh-8566ab23

Portfolio contents

- Case study README with results for every agent version
- All 21 agent instruction files and the shared house-style.md
- Test inputs and answer keys for every stage
- `examples/refresh-from-excel/`: one full run with real outputs at every stage
- A step-by-step learning journal

Built in Claude Projects. All product names in the test material are fictional (Contoso Charts).

SINGLE SOURCE OF TRUTH

Appendix A: The shared house style

The 23 rules shared by the planner, drafter, reviewer, and QA agent (house-style.md v1.1), based on the Microsoft Writing Style Guide with house decisions.

```
# House style (single source of truth)

Version: 1.1
Based on: the Microsoft Writing Style Guide, with house decisions.
Used by: Doc planner (titles), Doc drafter (to write), and Style guide reviewer (to check).
Change a rule HERE, never in an agent's instructions.

## Voice and tone
1. Second person. Address the reader as "you."
   Bad: "The user should select the file."  Good: "Select the file."
2. Active voice, present tense.
   Bad: "The file will be saved by the system."  Good: "The system saves the file."
   A sentence that describes a UI state isn't passive.
   OK: "The check box is selected by default."
3. Contractions are encouraged.
   Bad: "You do not need to restart."  Good: "You don't need to restart."
4. No "we," "let us," or "let's" for the writer's voice.
   Bad: "Let us create a simple chart."  Good: "Create a simple chart."
5. No filler or condescension: "simply," "just," "easy," "obviously."
   Use "please" only when asking the reader to do something inconvenient.
   Bad: "It's simply a matter of following the steps."  Good: "Follow these steps."

## Headings
6. Sentence-style capitalization in titles and headings.
   Bad: "Configure Your Network Settings"  Good: "Configure your network settings"
7. Task headings start with a verb. No questions, no "How to."
   Bad: "How to uninstall?"  Good: "Uninstall Contoso Charts"
   Exception: a Troubleshooting title describes the symptom in the reader's
   words, and its section headings state causes.
   OK: "Charts don't update after editing Excel"

## Procedures and UI
8. UI actions: use "select" (not "click" or "click on"), "enter" (not "type in").
   Bold UI labels.
   Bad: "Click on the Save button."  Good: "Select **Save**."
9. One action per step. Say where before saying what.
   Bad: "Click Save on the File menu."  Good: "On the **File** menu, select **Save**."
   Standard dialog tasks count as ONE action. Don't split them.
   OK as one step: "In the **Save As** dialog box, choose a location and enter
   a file name."
10. Results aren't steps. Put a result in the same step, after the action.
    Bad: "4. Select **Install**. 5. The installation is indicated by a progress bar."
    Good: "4. Select **Install**. A progress bar shows the installation status."
11. No directional words for images or content: "above," "below," "as shown below."
    Bad: "Click Next as shown in the image below:"
    Good: "Select **Next**." (Place the image right after the step.)

## Word choice
12. Use "sign in" and "sign out," not "log in," "login," or "log out."
13. Bias-free language. Use "allow list/block list," "primary/replica,"
    and "they" or "you" instead of "he/she."
14. No Latin abbreviations: replace "e.g." with "for example," "i.e." with
    "that is," "via" with "through" or "by using." Avoid "etc."
15. Plain words: "use" not "utilize," "start" not "initiate,"
    "complete" or "fill in" not "fill up."
16. Word list: "email" (not "e-mail"); "double-click" (verb, hyphenated);
    "set up" (verb) vs. "setup" (noun).
    Bad: "Run the set up file."  Good: "Run the setup file."
17. "Might" for possibility, "can" for ability, "may" only for permission.
18. Versions: always name the product with a version number.
    Bad: "In version 4.2, ..."  Good: "In Contoso Charts 4.2, ..."

## Mechanics
19. Numbers: spell out zero through nine; use numerals for 10 and above.
    Technical values (versions, sizes, resolutions) always use numerals.
20. Serial (Oxford) comma: in a list of three or more items, put a comma
    before "and" or "or."
    Bad: "Select Name, Email and Phone."  Good: "Select Name, Email, and Phone."
21. Short sentences (aim for under 25 words). Put the key point first.
```

```
    If applying another rule leaves a sentence with no purpose, rewrite or
    delete it.
    Bad: "Contoso Charts provides an installation process."
    Good: "This article explains how to install and uninstall Contoso Charts."

## Accessibility
22. Every image needs alt text that describes what it shows.
    Bad: <img src="2.jpg">
    Good: <img src="2.jpg" alt="Setup wizard Welcome screen with the Next button">
23. Link text describes the destination. Never use "click here" or "here."
    Bad: "To know more on configuring the data grids click here."
    Good: "For more information, see Configure data grids."

## Shared markers (Doc drafter → Style guide reviewer)
- [S1], [S2] ... : source tags
- [GAP: ...] : missing information
- [CONFLICT: ...] : sources that disagree
```